Threat Modelling Report

Threat Model Summary:

Not Started	21
Not Applicable	0
Needs Investigation	0
Mitigation Implemented	11
Total	32
Total Migrated	0

Diagram: Diagram 1

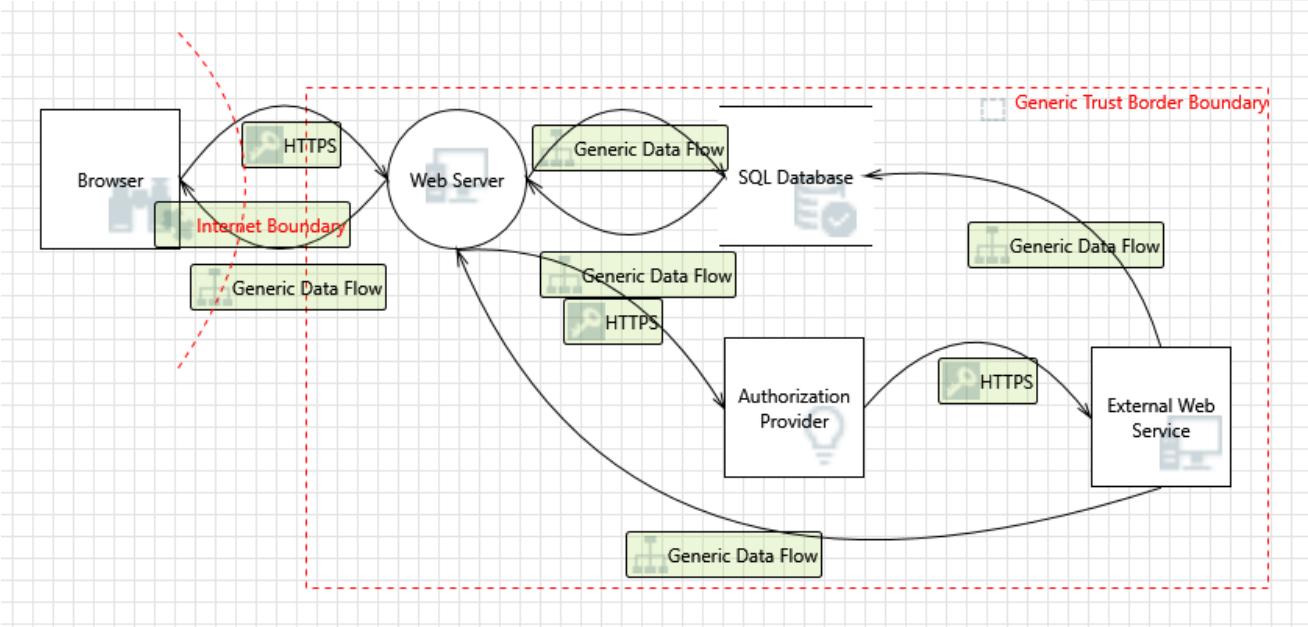
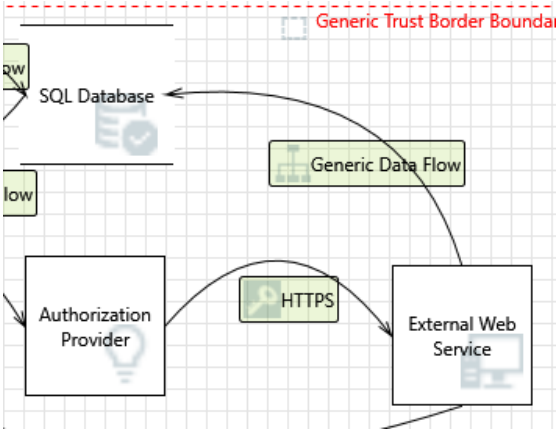


Diagram 1 Diagram Summary:

Not Started	21
Not Applicable	0
Needs Investigation	0
Mitigation Implemented	11
Total	32
Total Migrated	0

Interaction: Generic Data Flow



1. Spoofing of Destination Data Store SQL Database [State: Mitigation Implemented] [Priority: Low]

Category:	Spoofing
Description:	SQL Database may be spoofed by an attacker and this may lead to data being written to the attacker's target instead of SQL Database. Consider using a standard authentication mechanism to identify the destination data store.
Justification:	Not use hardcoded connection strings with plaintext passwords and Encrypt connection strings in configuration files.

2. Possible SQL Injection Vulnerability for SQL Database [State: Mitigation Implemented] [Priority: Low]

Category:	Tampering
Description:	SQL injection is an attack in which malicious code is inserted into strings that are later passed to an instance of SQL Server for parsing and execution. Any procedure that constructs SQL statements should be reviewed for injection vulnerabilities because SQL Server will execute all syntactically valid queries that it receives. Even parameterized data can be manipulated by a skilled and determined attacker.
Justification:	Use Parameterized Queries exclusively, utilize Stored Procedures with strict input validation and apply the Principle of Least Privilege to database accounts.

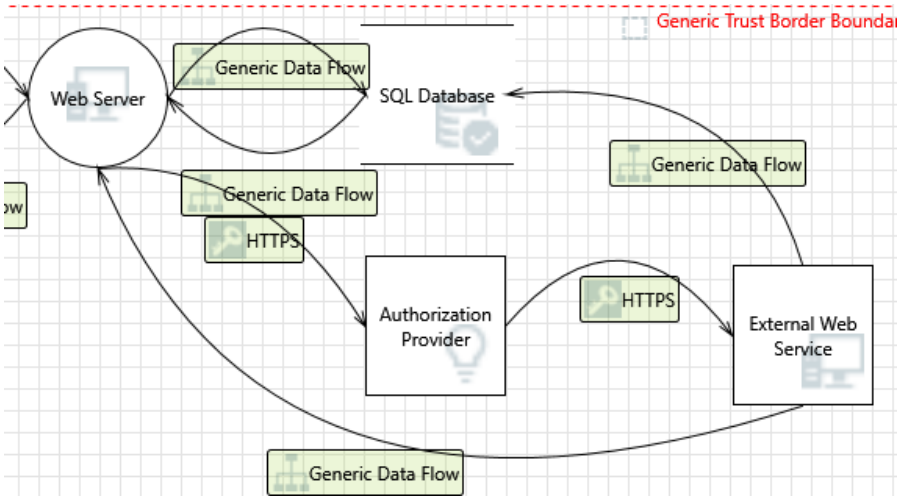
3. Lower Trusted Subject Updates Logs [State: Mitigation Implemented] [Priority: Low]

Category:	Repudiation
Description:	If you have trust levels, is anyone other outside of the highest trust level allowed to log? Letting everyone write to your logs can lead to repudiation problems. Only allow trusted code to log.
Justification:	Enforce Write-Once Read-Many (WORM) storage policies for log files. Use append-only data structures or SIEM solutions with cryptographic hashing to prevent log tampering.

4. Data Logs from an Unknown Source [State: Mitigation Implemented] [Priority: Low]

Category:	Repudiation
Description:	Do you accept logs from unknown or weakly authenticated users or systems? Identify and authenticate the source of the logs before accepting them.
Justification:	Require Mutual TLS (mTLS) for log shipping. Validate that log entries are digitally signed by the source system to ensure non-repudiation and integrity.

Interaction: Generic Data Flow



5. Spoofing the External Web Service External Entity [State: Mitigation Implemented] [Priority: Low]

Category: Spoofing

Description: External Web Service may be spoofed by an attacker and this may lead to unauthorized access to Web Server. Consider using a standard authentication mechanism to identify the external entity.

Justification: Implement Mutual TLS (mTLS) for service-to-service communication. Validate server certificates strictly. Use API Keys or OAuth 2.0 Client Credentials flow with secure secret storage (e.g., Azure Key Vault).

6. Cross Site Scripting [State: Not Started] [Priority: High]

Category: Tampering

Description: The web server 'Web Server' could be a subject to a cross-site scripting attack because it does not sanitize untrusted input.

Justification: <no mitigation provided>

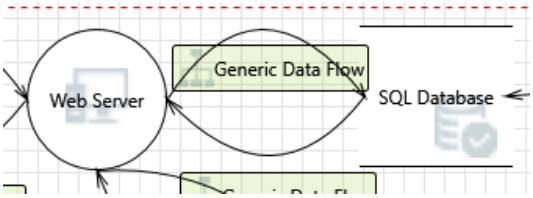
7. Elevation Using Impersonation [State: Not Started] [Priority: High]

Category: Elevation Of Privilege

Description: Web Server may be able to impersonate the context of External Web Service in order to gain additional privilege.

Justification: <no mitigation provided>

Interaction: Generic Data Flow



8. Spoofing of Destination Data Store SQL Database [State: Mitigation Implemented] [Priority: Low]

Category: Spoofing

Description: SQL Database may be spoofed by an attacker and this may lead to data being written to the attacker's target instead of SQL Database. Consider using a standard authentication mechanism to identify the destination data store.

Justification: Not use hardcoded connection strings with plaintext passwords and Encrypt connection strings in configuration files

9. Potential SQ Injection Vulnerability for SQL Database [State: Mitigation Implemented] [Priority: Low]

Category: Tampering

Description: SQL injection is an attack in which malicious code is inserted into strings that are later passed to a n instance of SQL Server for parsing and execution. Any procedure that constructs SQL statements should be reviewed for injection vulnerabilities because SQL Server will execute all syntactically valid queries that it receives. Even parameterized data can be manipulated by a skilled and determined attacker.

Justification: Use an ORM (Object Relational Mapper) like Entity Framework which parameterizes queries by default. Sanitize all inputs using whitelist validation.

10. Potential Excessive Resource Consumption for Web Server or SQL Database [State: Not Started] [Prio ity: High]

Category: Denial Of Service

Description: Does Web Server or SQL Database take explicit steps to control resource consumption? Resource consumption attacks can be hard to deal with, and there are times that it makes sense to let the OS do the job. Be careful that your resource requests don't deadlock, and that they do timeout.

Justification: <no mitigation provided>

11. Data Logs from an Unknown Source [State: Not Started] [Priority: High]

Category: Repudiation

Description: Do you accept logs from unknown or weakly authenticated users or systems? Identify and authenticate the source of the logs before accepting them.

Justification: <no mitigation provided>

12. Lower Trusted Subject Updates Logs [State: Not Started] [Priority: High]

Category: Repudiation

Description: If you have trust levels, is anyone other outside of the highest trust level allowed to log? Letting everyone write to your logs can lead to repudiation problems. Only allow trusted code to log.

Justification: <no mitigation provided>

13. Risks from Logging [State: Not Started] [Priority: High]

Category: Tampering

Description: Log readers can come under attack via log files. Consider ways to canonicalize data in all logs. Implement a single reader for the logs, if possible, in order to reduce attack surface area. Be sure to understand and document log file elements which come from untrusted sources.

Justification: <no mitigation provided>

14. Weak Credential Storage [State: Mitigation Implemented] [Priority: Low]

Category: Information Disclosure

Description: Credentials held at the server are often disclosed or tampered with and credentials stored on the client are often stolen. For server side, consider storing a salted hash of the credentials instead of storing the credentials themselves. If this is not possible due to business requirements, be sure to encrypt the credentials before storage, using an SDL-approved mechanism. For client side, if storing credentials is required, encrypt them and protect the data store in which they're stored

Justification: Used a dedicated secrets management solution

15. Insufficient Auditing [State: Not Started] [Priority: High]

Category: Repudiation

Description: Does the log capture enough data to understand what happened in the past? Do your logs capture enough data to understand an incident after the fact? Is such capture lightweight enough to be left on all the time? Do you have enough data to deal with repudiation claims? Make sure you log sufficient and appropriate data to handle a repudiation claims. You might want to talk to an audit expert as well as a privacy expert about your choice of data.

Justification: <no mitigation provided>

16. Potential Weak Protections for Audit Data [State: Not Started] [Priority: High]

Category: Repudiation

Description: Consider what happens when the audit mechanism comes under attack, including attempts to destroy the logs, or attack log analysis programs. Ensure access to the log is through a reference monitor, which controls read and write separately. Document what filters, if any, readers can rely on, or writers should expect

Justification: <no mitigation provided>

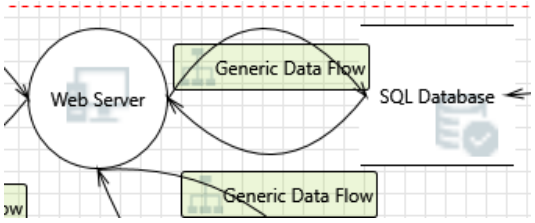
17. Authorization Bypass [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: Can you access SQL Database and bypass the permissions for the object? For example by editing the files directly with a hex editor, or reaching it via filesharing? Ensure that your program is the only one that can access the data, and that all other subjects have to use your interface.

Justification: <no mitigation provided>

Interaction: Generic Data Flow



18. Weak Access Control for a Resource [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: Improper data protection of SQL Database can allow an attacker to read information not intended for disclosure. Review authorization settings.

Justification: <no mitigation provided>

19. Persistent Cross Site Scripting [State: Not Started] [Priority: High]

Category: Tampering

Description: The web server 'Web Server' could be a subject to a persistent cross-site scripting attack because it does not sanitize data store 'SQL Database' inputs and output.

Justification: <no mitigation provided>

20. Cross Site Scripting [State: Not Started] [Priority: High]

Category: Tampering

Description: The web server 'Web Server' could be a subject to a cross-site scripting attack because it does not sanitize untrusted input.

Justification: <no mitigation provided>

21. Spoofing of Source Data Store SQL Database [State: Mitigation Implemented] [Priority: Low]

Category: Spoofing

Description: SQL Database may be spoofed by an attacker and this may lead to incorrect data delivered to Web Server. Consider using a standard authentication mechanism to identify the source data store.

Justification: Not use hardcoded connection strings with plaintext passwords and Encrypt connection strings in configuration files.

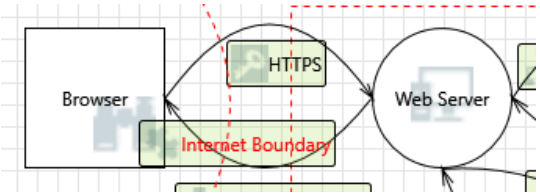
22. Risks from Logging [State: Not Started] [Priority: High]

Category: Tampering

Description: Log readers can come under attack via log files. Consider ways to canonicalize data in all logs. Implement a single reader for the logs, if possible, in order to reduce attack surface area. Be sure to understand and document log file elements which come from untrusted sources.

Justification: <no mitigation provided>

Interaction: HTTPS



23. Spoofing the Browser External Entity [State: Mitigation Implemented] [Priority: Low]

Category: Spoofing

Description: Browser may be spoofed by an attacker and this may lead to unauthorized access to Web Server. Consider using a standard authentication mechanism to identify the external entity.

Justification: Implement Multi-Factor Authentication (MFA) and strong password policies. Ensure HTTPS (TLS 1.2+) is strictly enforced with valid certificates to prevent credential interception.

24. Cross Site Scripting [State: Not Started] [Priority: High]

Category: Tampering

Description: The web server 'Web Server' could be a subject to a cross-site scripting attack because it does not sanitize untrusted input.

Justification: <no mitigation provided>

25. Elevation Using Impersonation [State: Not Started] [Priority: High]

Category: Elevation Of Privilege

Description: Web Server may be able to impersonate the context of Browser in order to gain additional privilege.

Justification: <no mitigation provided>

26. Potential Data Repudiation by Web Server [State: Not Started] [Priority: High]

Category: Repudiation

Description: Web Server claims that it did not receive data from a source outside the trust boundary. Consider using logging or auditing to record the source, time, and summary of the received data.

Justification: <no mitigation provided>

27. Potential Process Crash or Stop for Web Server [State: Not Started] [Priority: High]

Category: Denial Of Service

Description: Web Server crashes, halts, stops or runs slowly; in all cases violating an availability metric.

Justification: <no mitigation provided>

28. Data Flow HTTPS Is Potentially Interrupted [State: Not Started] [Priority: High]

Category: Denial Of Service

Description: An external agent interrupts data flowing across a trust boundary in either direction.

Justification: <no mitigation provided>

29. Web Server May be Subject to Elevation of Privilege Using Remote Code Execution [State: Not Started] [Priority: High]

Category: Elevation Of Privilege

Description: Browser may be able to remotely execute code for Web Server.

Justification: <no mitigation provided>

30. Elevation by Changing the Execution Flow in Web Server [State: Not Started] [Priority: High]

Category: Elevation Of Privilege

Description: An attacker may pass data into Web Server in order to change the flow of program execution within Web Server to the attacker's choosing.

Justification: <no mitigation provided>

31. Cross Site Request Forgery [State: Not Started] [Priority: High]

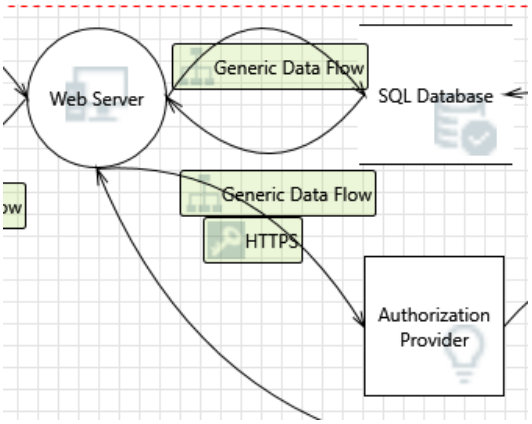
Category: Elevation Of Privilege

Description: Cross-site request forgery (CSRF or XSRF) is a type of attack in which an attacker forces a user's browser to make a forged request to a vulnerable site by exploiting an existing trust relationship

between the browser and the vulnerable web site. In a simple scenario, a user is logged in to web site A using a cookie as a credential. The other browses to web site B. Web site B returns a page with a hidden form that posts to web site A. Since the browser will carry the user's cookie to web site A, web site B now can take any action on web site A, for example, adding an admin to an account. The attack can be used to exploit any requests that the browser automatically authenticates, e.g. by session cookie, integrated authentication, IP whitelisting. The attack can be carried out in many ways such as by luring the victim to a site under control of the attacker, getting the user to click a link in a phishing email, or hacking a reputable web site that the victim will visit. The issue can only be resolved on the server side by requiring that all authenticated state-changing requests include an additional piece of secret payload (canary or CSRF token) which is known only to the legitimate web site and the browser and which is protected in transit through SSL/TLS. See the Forgery Protection property on the flow stencil for a list of mitigations.

Justification: <no mitigation provided>

Interaction: HTTPS



32. Weakness in SSO Authorization [State: Mitigation Implemented] [Priority: Low]

Category: Elevation Of Privilege

Description: Common SSO implementations such as OAUTH2 and OAUTH Wrap are vulnerable to MitM attacks.

Justification: Enforce HTTPS with HSTS. Validate the ‘state’ and ‘nonce’ parameters in OAuth 2.0/OIDC flows to prevent CSRF and replay attacks. Use PKCE for public clients